

FitTrack Pro - Code Listing (selected files)

```
--- app.py ---
from flask import Flask
from flask_cors import CORS
from routes.auth import auth_bp
from routes.users import users_bp
from routes.workouts import workouts_bp
from routes.measurements import measurements_bp
from routes.goals import goals_bp
from routes.exercises import exercises_bp
from utils.db import get_db

def create_app():
    app = Flask(__name__)
    app.config.from_mapping({
        "MONGO_URI": __import__('os').environ.get("MONGO_URI", "mongodb://localhost:27017/fittrack_db"),
        "JWT_SECRET_KEY": __import__('os').environ.get("JWT_SECRET_KEY", "change-this-secret"),
    })
    CORS(app)
    get_db(app) # initialize DB client

    app.register_blueprint(auth_bp, url_prefix="/auth")
    app.register_blueprint(users_bp, url_prefix="/users")
    app.register_blueprint(workouts_bp, url_prefix="/workouts")
    app.register_blueprint(measurements_bp, url_prefix="/measurements")
    app.register_blueprint(goals_bp, url_prefix="/goals")
    app.register_blueprint(exercises_bp, url_prefix="/exercises")

    @app.route("/")
    def index():
        return {"app": "FitTrack Pro backend", "status": "ok"}

    return app

if __name__ == "__main__":
    app = create_app()
    app.run(port=5000)
--- routes/auth.py ---
from flask import Blueprint, request, current_app, jsonify
from werkzeug.security import generate_password_hash, check_password_hash
from flask_jwt_extended import create_access_token, JWTManager
from utils.db import get_db

auth_bp = Blueprint('auth', __name__)
jwt = None

@auth_bp.record_once
def on_load(state):
    global jwt
    app = state.app
    jwt = JWTManager(app)

@auth_bp.route('/register', methods=['POST'])
def register():
    db = get_db()
    users = db.get_default_database().users
    body = request.get_json() or {}
    email = body.get('email')
    password = body.get('password')
    name = body.get('name')
    if not email or not password:
        return {"msg": "email and password required"}, 400
    if users.find_one({'email': email}):
        return {"msg": "user exists"}, 409
    pw_hash = generate_password_hash(password)
    doc = {'email': email, "password_hash": pw_hash, "name": name, "created_at": __import__('datetime').datetime.now()}
    res = users.insert_one(doc)
    user = users.find_one({'_id': res.inserted_id, {'password_hash': 0}})
    return jsonify({"user": {"id": str(res.inserted_id), "email": email, "name": name}}), 201
```

```

@auth_bp.route('/login', methods=['POST'])
def login():
    db = get_db()
    users = db.get_default_database().users
    body = request.get_json() or {}
    email = body.get('email')
    password = body.get('password')
    if not email or not password:
        return {"msg": "email and password required"}, 400
    user = users.find_one({'email': email})
    if not user or not check_password_hash(user.get('password_hash', ''), password):
        return {"msg": "invalid credentials"}, 401
    access = create_access_token(identity=str(user['_id']))
    return {"access_token": access}, 200
--- routes/workouts.py ---
from flask import Blueprint, request, jsonify
from utils.db import get_db
from utils.jwt_auth import jwt_required, current_user_id
from bson.objectid import ObjectId
import datetime

```

```
workouts_bp = Blueprint('workouts', __name__)
```

```

@workouts_bp.route("", methods=['POST'])
@jwt_required
def create_workout():
    db = get_db().get_default_database()
    body = request.get_json() or {}
    uid = current_user_id()
    doc = {
        'user_id': ObjectId(uid),
        'type': body.get('type'),
        'start_time': body.get('start_time') or datetime.datetime.utcnow().isoformat(),
        'end_time': body.get('end_time'),
        'duration_minutes': body.get('duration_minutes'),
        'distance_km': body.get('distance_km', 0),
        'calories_burned': body.get('calories_burned', 0),
        'route': body.get('route', []),
        'exercises': [],
        'notes': body.get('notes', ''),
        'created_at': datetime.datetime.utcnow()
    }
    res = db.workouts.insert_one(doc)
    doc['_id'] = str(res.inserted_id)
    return jsonify(doc), 201

```

```

@workouts_bp.route("", methods=['GET'])
@jwt_required
def list_workouts():
    db = get_db().get_default_database()
    uid = current_user_id()
    q = {'user_id': ObjectId(uid)}
    # filters
    t = request.args.get('type')
    if t:
        q['type'] = t
    start = request.args.get('start')
    end = request.args.get('end')
    if start or end:
        q['start_time'] = {}
        if start:
            q['start_time']['$gte'] = start
        if end:
            q['start_time']['$lte'] = end
    cur = db.workouts.find(q).sort('start_time', -1).limit(200)
    out = []
    for w in cur:
        w['_id'] = str(w['_id'])
        w['user_id'] = str(w['user_id'])
        out.append(w)
    return jsonify(out)

```

```

@workouts_bp.route('/<id>', methods=['GET'])
@jwt_required
def get_workout(id):
    db = get_db().get_default_database()
    try:
        w = db.workouts.find_one({'_id': ObjectId(id)})
    except Exception:
        return {'msg': 'invalid id'}, 400
    if not w:
        return {'msg': 'not found'}, 404
    w['_id'] = str(w['_id'])
    w['user_id'] = str(w['user_id'])
    return jsonify(w)

@workouts_bp.route('/<id>', methods=['PUT'])
@jwt_required
def update_workout(id):
    db = get_db().get_default_database()
    body = request.get_json() or {}
    allowed = {'type', 'start_time', 'end_time', 'duration_minutes', 'distance_km', 'calories_burned', 'notes'}
    update = {k: body[k] for k in body if k in allowed}
    if not update:
        return {'msg': 'nothing to update'}, 400
    res = db.workouts.update_one({'_id': ObjectId(id)}, {'$set': update})
    if res.matched_count == 0:
        return {'msg': 'not found'}, 404
    return {'msg': 'updated'}, 200

@workouts_bp.route('/<id>', methods=['DELETE'])
@jwt_required
def delete_workout(id):
    db = get_db().get_default_database()
    res = db.workouts.delete_one({'_id': ObjectId(id)})
    if res.deleted_count == 0:
        return {'msg': 'not found'}, 404
    return "", 204

@workouts_bp.route('/analytics/weekly_distance', methods=['GET'])
@jwt_required
def weekly_distance():
    # aggregation: total distance per day for last 7 days
    db = get_db().get_default_database()
    uid = current_user_id()
    pipeline = [
        {'$match': {'user_id': ObjectId(uid)}},
        {'$project': {'distance_km': 1, 'date': {'$dateFromString': {'dateString': '$start_time'}}}},
        {'$group': {'_id': {'$dateToString': {'format': '%Y-%m-%d', 'date': '$date'}}, 'total': {'$sum': '$distance_km'}}},
        {'$sort': {'_id': 1}}
    ]
    res = list(db.workouts.aggregate(pipeline))
    return jsonify(res)

```